

findings

Search-flow root-cause audit — 2026-06-15

Revision: 1 **Last modified:** 2026-06-15T00:00:00Z

Code-only audit (no running stack). BUG = provable from code;
NEEDS-STACK = suspected, needs runtime confirm.

End-to-end path

1. Dashboard search() ->
api.search({query,limit,sort_by,sort_order}) -> POST /api/v1/search (dashboard.component.ts:336, api.service.ts:17). Body model has NO provider field (search.model.ts:1-9).
2. FastAPI POST /search (routes.py:275) -> orch.start_search, spawns background_run_search, returns status:"running"+stream_token.
3. Dashboard opens SSE GET /api/v1/search/stream/{id} (sse.service.ts:24, dashboard.component.ts:349).
4. _run_search (search.py:718) fans out over _get_enabled_trackers() (search.py:924) — server decides provider set; client cannot constrain.
5. SSE streamer (streaming.py:112) emits result_found, merged_update, tracker_started/completed, results_update, search_complete.
6. On search_complete dashboard calls GET /api/v1/search/{id} (get_search, routes.py:568). Go mirrors in internal/api/search.go + internal/service/merge_search.go.

BUG-1 No provider-selection parameter exists (single/selected modes unimplemented) [BUG, highest impact]

Files: routes.py:113-120 (SearchRequest: query,category,limit,enable_metadata,validate_trackers,sort_by,sort_order — no trackers/providers/sources); search.model.ts:1-9 (same); search.py:924-945 (_get_enabled_trackers takes no selection); search.py:662/686/718/743 (calls with no filter); Go models/search.go:3-8 (no providers); service/merge_search.go:190 (StartSearch(query,[string{"all"},category)). Root cause: only "search all enabled providers" exists. No wire field, no orchestrator param, no plugin filter for single/subset. Single/selected flows have nothing to exercise -> silently behave as "all". Biggest reason single/selected modes "do not work". Fix: add trackers:list[str]|None to SearchRequest + Angular model; thread into start_search/_run_search/search -> _get_enabled_trackers(selected); filter t.name in selected (validate_tracker_name); None/empty=all (back-compat). Go: add Trackers[string, map to qbitClient.StartSearch plugins. RED test: tests/unit/merge_service/test_provider_selection.py::test_single_provider_filters_fanout — monkeypatch _search_tracker to record tracker.name, enable >=3 trackers, await orch.search(query="x",trackers=["rutracker"]); ASSERT recorded set=={"rutracker"} and metadata.trackers_searched==["rutracker"]. Today fails: search() rejects trackers= kwarg.

BUG-2 Dashboard SSE service never forwards merged_update (live merged grid dead) [BUG, high impact]

Files: sse.service.ts:30-58 (addEventListener only for search_start,result_found,results_update,search_complete,download_*; NONE for merged_update/tracker_started/tracker_completed); backend emits all (streaming.py:249,310,195,201); dashboard handles them (dashboard.component.ts:367,377,380). Root cause: EventSource dispatches a NAMED event only to a listener for that exact name; unregistered named events are silently dropped (no fallback to onmessage). So merged_update/tracker_started/tracker_completed case branches never reached. Progressive deduplicated grid (liveMergedResults, :371) never updates during search; tracker chips never flip. Final grid still appears via search_complete->loadSearchResults(GET), but live experience broken; remote/slow-GET shows perpetual empty "Searching...". Fix: add the 3 listeners in sse.service.ts mirroring existing pattern. RED test: sse.service.spec.ts::forwards merged_update

events — stub EventSource, dispatch named merged_update
data='{ "merged_results":2,"results":[{"name":"X"}] }', subscribe
sse.events; ASSERT emission {event:'merged_update',data:
{merged_results:2}}. Today no emission. Sibling cases for
tracker_started/completed.

BUG-3 Go RunSearch re-appends offset-0 results every tick -> duplicated, ever-growing; never merged [BUG, high impact, Go]

Files: service/merge_search.go:206-239 — 500ms ticker calls
GetSearchResults(searchIDInt,100,0) (fixed offset 0) and appends
ALL rows every tick (:218-228); SetMergedResults (:163) never
called -> MergedResults stays 0. Root cause: offset never
advances, prior results never replaced -> list grows by up to 100
dup rows/500ms until stop; nothing merges. SSE results
(search.go:128) + SearchSyncHandler (search.go:71) stream dup-
laden unmerged list. Fix: per-search seen-set (FileURL/infohash)
or replace slice with latest snapshot each tick; dedup pass +
SetMergedResults before completed; set meta.MergedResults.
RED test: service/
service_test.go::TestRunSearch_NoDuplicateAccumulation —
httpTest qbit stub returns SAME single result each poll, Running
x3 then Stopped; ASSERT len(GetLiveResults)==1 and
MergedResults==1. Today ~3-4 copies, MergedResults==0.

BUG-4 Go GetSearchHandler (final GET) never returns results [BUG, high impact, Go]

Files: api/search.go:144-165 — SearchResponse omits Results (no
GetLiveResults/GetMergedResults). GetMergedResults has zero
callers. Root cause: the GET-by-id the dashboard hits on
search_complete (dashboard.component.ts:387->api.getSearch)
returns empty results. Go-profile grid empty after every search.
Fix: merged, _ := svc.GetMergedResults(searchID) (fallback
GetLiveResults); set Results: merged. RED test: api/
api_test.go::TestGetSearch_ReturnsResults — seed completed
search w/ 2 merged via SetMergedResults, GET /api/v1/search/
{id}; ASSERT len(resp.Results)==2. Today empty.

BUG-5 Remote-instance: SSE/API base URL hardcoded-relative; no remote base + CORS gap on SSE [NEEDS-STACK, high impact for remote instance]

Files: `api.service.ts:15` (`baseUrl=''`); `sse.service.ts:24` (`relative'/api/v1/search/stream/'+id`; new `EventSource`, no `withCredentials`). Root cause (suspected): dashboard served from a different origin than API -> POST/GET + `EventSource` hit the page origin not the remote API -> 404/blank. Even if `baseUrl` set, `EventSource` needs absolute URL AND server CORS must allow the SSE origin (+`withCredentials` if token/cookie). Matches "does not work from a remote API instance". Relative-URL hardcoding is provable; exact failure (404 vs CORS vs blank) NEEDS-STACK. Fix: single configurable API base (environment/runtime config / `PUBLIC_HOST` per `CLAUDE.md` no-hardcoded-host); use for HTTP + absolute `EventSource` URL; verify Go/Python CORS allows dashboard origin on SSE route. RED test: `sse.service.spec.ts::uses configured base for stream URL` — set API base `https://remote.example:7187`, `connect('abc')`, capture `EventSource` ctor URL; `ASSERT` starts with `https://remote.example:7187/api/v1/search/stream/abc`. Today relative -> fails.

BUG-6 _update_best_quality fallback crashes on integer size (None-crash sibling) -> aborts whole merge -> zero results [BUG conditional, medium-high]

Files: `deduplicator.py:155-182` (`_update_best_quality`) on `ImportError` of `api.routes._detect_quality` (:158-160) calls `_fallback_quality(r.name,r.size)` (:176); `_fallback_quality` (:506-538)-> `_parse_size_to_bytes(size)` (:529); `_parse_size_to_bytes` (:540-563) does `re.match(...,size_str,re.I)` with `NO str()` coercion (:547); `merge_results` (:90) calls `_update_best_quality` with no `try/except`; called from `_run_search` (`search.py:856`). Root cause: plugins emit size as `int` byte-count (-1 sentinel). Project already fixed identical `None/int` crash in `_parse_size` (:301-337 docstring) and `routes._parse_size_to_bytes` (uses `str()`). Static `_parse_size_to_bytes` in `deduplicator.py` still passes raw `int` into `re.match` -> `TypeError`. Reproduced: `re.match(r"([+])(GB|MB...)",4096,re.I)` -> `TypeError` expected string or bytes-like object. Reached when `api.routes import` raises `ImportError`; then unguarded `_update_best_quality` propagates -> `merge_results` -> `_run_search` outer `except` (:865) flips `status="failed"`, all results dropped. `ImportError` branch

frequency NEEDS-STACK; crash itself proven. Fix: coerce in deduplicator._parse_size_to_bytes: if not size_str return 0 then str(size_str) (mirror routes); optionally wrap _update_best_quality loop. RED test: tests/unit/merge_service/test_deduplicator.py::test_merge_survives_integer_size — SearchResults size=4096(int) and -1, force fallback (monkeypatch api.routes import to raise OR call _fallback_quality("Movie 1080p",4096)); ASSERT merge_results returns non-empty list (today raises TypeError).

BUG-7 All-tracker failures swallowed; async POST /search reports completed/ 0 with no actionable signal [BUG, medium]

Files: search.py:947-985 (_search_tracker outer except :982 logs+returns []; private-tracker errors fully swallowed); :816-821 (_search_one records stat.error but status->"completed" :859 if merge ok even on 0 rows); _search_rutracker :1151 returns [] silently on missing creds. CAPTCHA 403 special-case only in search_sync (routes.py:455-473), NOT in async POST /search the dashboard uses. Root cause: every-provider-fail (expired cookies, missing creds, CAPTCHA, network) completes with status:"completed",total_results:0 -> dashboard "No results found." (dashboard.component.ts:389), indistinguishable from genuine empty. No "fix auth/CAPTCHA" signal -> looks like "search doesn't work". Fix: when total_results==0 and errors/tracker_stats show auth/captcha/error, set distinct status (no_results vs auth_required); surface errors/tracker_stats in search_complete; render per-tracker error chips (blocked by BUG-2). RED test: tests/unit/merge_service/test_search_errors.py::test_all_trackers_error_is_distinguishable — monkeypatch _search_tracker to raise for every enabled tracker, await orch.search("x"); ASSERT metadata.status!="completed" and len(metadata.errors)==N. Today completed w/ empty errors.

Notes / non-bugs

- Prior None-seed sort fix (deduplicator.py:71) and _parse_size int handling (:301-337) correct. BUG-6 is the remaining sibling in static _parse_size_to_bytes.
- _qbit_login_succeeded/_qbit_add_succeeded (routes.py:747-815) are download-path, correct for modern qBittorrent.
- Python "all providers" path fans out correctly. Breakage concentrates in single/selected (BUG-1), live stream (BUG-2),

Go profile (BUG-3/4), remote-instance (BUG-5), silent-failure
UX (BUG-7).